

Чек-лист доработок до идеального MVP

Проект: интерфейс управления жизненным циклом рабочего места программиста

Основано на текущем архиве проекта и на ТЗ. Пункт __rusache__ сознательно не включен: пользователь указал, что с этим разберется отдельно.

Приоритет	Смысл
P0	Критично для ощущения рабочей программы и соответствия MUST-требованиям.
P1	Нужно, чтобы MVP был почти полностью закрыт по ТЗ и нормально выглядел на защите.
P2	Полировка, документация и качество реализации; не обязательно для первого запуска, но желательно для “идеального” состояния.

Краткий вывод

До “100% MVP” главная недостака не в идее, а в пользовательских сценариях и строгой валидации модели. Нужно, чтобы пользователь мог не только смотреть демо, но и руками создать проект, артефакты, связи, фрагмент кода, затем увидеть это в графе, анализе влияния и экспорте.

Полный список доработок

Приоритет	Блок	Что сделать	Зачем
P0	Frontend: проекты	Добавить создание проекта через UI: форма с названием и описанием.	Сейчас пользователь в основном выбирает существующий проект или грузит демо; по ТЗ проект должен создаваться из интерфейса.
P0	Frontend: артефакты	Сделать рабочую кнопку “Создать артефакт”: модальное окно/страница с типом, кодом, названием, описанием, статусом, приоритетом.	Закрывает базовый CRUD и главный сценарий ведения реестра артефактов.
P0	Frontend: артефакты	Добавить редактирование и удаление артефакта из UI с подтверждением удаления.	API частично есть, но пользовательский сценарий должен быть виден на демонстрации.
P0	Frontend: карточка	Сделать карточку артефакта: сведения, метаданные, входящие/исходящие связи, код, действия редактирования.	Артефакт должен восприниматься как полноценная единица жизненного цикла, а не только строка таблицы.
P0	Frontend: связи	Добавить создание связи через UI: источник, тип связи, цель, комментарий/label.	Ключевая функция темы: пользователь сам строит граф/онтологическую модель.
P0	Frontend: связи	Добавить список и удаление входящих/исходящих связей в карточке артефакта или отдельном экране.	Без этого связь сложно проверить и сопровождать.
P0	Frontend: код	Добавить редактор/просмотр кода для артефакта File/Function/Module через Monaco или хотя бы textarea.	В ТЗ код является отдельным артефактом жизненного цикла; зависимость Monaco уже есть, но сценарий должен быть в UI.
P0	Backend: артефакты	Проверять существование project_id и type_id при создании/обновлении артефакта.	Иначе можно создать некорректные записи, которые ломают онтологическую модель.
P0	Backend: артефакты	Запретить дублирование code внутри одного проекта: проверка в сервисе + UniqueConstraint(project_id, code).	Это прямо соответствует ТЗ и нужно для стабильных ссылок REQ-001, ENT-001 и т.д.
P0	Backend: связи	Проверять существование relation_type_id перед созданием связи.	Сейчас связь может ссылаться на несуществующий тип, если это не исправлено локально.
P0	Backend: связи	Запретить дубли одинаковых связей source-target-type в одном проекте.	Снижает мусор в графе и делает анализ влияния предсказуемым.
P0	Backend: удаление	При удалении артефакта удалять все входящие и исходящие ArtifactRelation либо блокировать удаление с предупреждением.	Иначе возможны сиротские связи и некорректный граф.
P1	Backend: ограничения графа	Реализовать проверку source_types_json и target_types_json для типов связей.	В модели поля есть, но для “100% по ТЗ” нужно реально запрещать недопустимые пары типов.
P1	Backend: циклы	Оставить запрет циклов для is_a и part_of, добавить тесты на эти случаи.	Это базовое онтологическое ограничение и хороший пункт для защиты.

Приоритет	Блок	Что сделать	Зачем
P1	Impact analysis	По умолчанию учитывать только связи с affects_impact=True; related_to/describes включать только по выбору пользователя.	Иначе анализ влияния превращается в обход всех связей подряд, включая слабые/документирующие.
P1	Impact analysis	Сделать понятный path: REQ-001 -> realizes <- MOD-001 -> part_of <- FILE-001, с промежуточными артефактами и направлениями.	Сейчас путь может быть слишком упрощен; на защите важно показать трассируемость.
P1	Impact analysis	Добавить выбор типов связей в интерфейсе анализа влияния.	В API поле include_relation_types есть, но пользователь должен уметь управлять областью анализа.
P1	Impact analysis	Подсвечивать найденные элементы на графе после анализа или дать ссылку “показать на графе”.	Сильно повышает демонстрационную ценность функции.
P1	Граф	Добавить фильтры графа по типам артефактов и типам связей.	FR-012 в ТЗ помечен SHOULD; для идеального MVP лучше закрыть.
P1	Граф	Сделать кликабельные узлы графа с переходом в карточку артефакта.	Граф должен быть не картинкой, а навигацией по модели проекта.
P1	Граф	Визуально различать типы артефактов и типы связей: цвет/иконка/легенда.	Это важно для восприятия онтологической структуры.
P1	Диагностика	Добавить отдельный блок диагностики: артефакты без связей, требования без тестов, требования без реализации, код без требования.	FR-020: сейчас частично есть только счетчик изолированных элементов.
P1	Поиск/фильтрация	Добавить поиск по коду/названию/описанию и фильтры по типу, статусу, приоритету в реестре артефактов.	FR-014 заявлен в ТЗ, в UI должен быть явно виден.
P1	Требования	Для Requirement добавить поля/метаданные: requirement_type, acceptance_criteria, rationale/source.	Иначе требования почти не отличаются от обычных артефактов.
P1	Тесты	Для TestCase добавить поля: test_type, preconditions, steps, expected_result.	Помогает показать связь “требование - тест” содержательно, а не только графически.
P1	Код	Разрешить просмотр/сохранение language, file_path, start_line/end_line; показывать текущий сохраненный фрагмент.	Закрывает CodeFragment как нормальную сущность, а не просто текстовое поле.
P1	Экспорт JSON	Добавить импорт JSON проекта либо честно перенести импорт в “перспективы развития”.	В ТЗ импорт JSON стоит как SHOULD; для 100% MVP его лучше реализовать.
P1	Экспорт RDF/Turtle	Расширить Turtle: проект, description, relation label/comment, типы классов/свойств, безопасные URI для кодов.	Сейчас экспорт минимален; для онтологической темы лучше сделать его богаче.
P1	ChangeLog	Либо реализовать журнал изменений для создания/редактирования/удаления артефактов и связей, либо убрать из заявлений.	Модель ChangeLog есть, но функция не закрыта; в ТЗ FR-015 SHOULD.
P1	ExportRecord	Фиксировать факты экспорта в export_records или убрать модель из описания реализации.	Сейчас модель выглядит заготовкой без использования.
P1	Backend: сервисы	Вынести бизнес-логику CRUD артефактов и проектов из роутеров в services.	Архитектура станет чище: роуты принимают запросы, сервисы проверяют правила.
P1	Backend: errors	Сделать единые сообщения ошибок и корректные HTTP-коды: 400/404/409.	Для демонстрации и тестов важно различать “не найдено”, “конфликт”, “некорректные данные”.
P1	Backend: schema	metadata_json = Column(JSON, default=dict), не default={}.	Небольшая, но правильная техническая правка.
P1	Backend: FK/cascade	Настроить cascade/onDelete для связей и код-фрагментов; добавить индексы на project_id, code, source/target.	Повышает целостность данных и качество схемы БД.
P1	Backend: config	Вынести DATABASE_URL и CORS origins в config/.env.example.	Сейчас локальные значения лучше не хардкодить; для README и запуска это аккуратнее.
P1	Backend: init	Убрать побочные эффекты из импорта main.py: создание таблиц и init_db выполнять через startup/app factory.	Иначе тесты и импорт приложения могут трогать рабочую БД.

Приоритет	Блок	Что сделать	Зачем
P1	Backend: tests	Добавить отдельную тестовую БД/in-memory fixture для всех тестов.	Тесты не должны зависеть от workplace.db.
P1	Frontend: API	Заменить const API_URL = http://localhost:8000 на import.meta.env.VITE_API_URL + .env.example.	Проект станет переносимее и аккуратнее.
P1	Frontend: UX	Добавить отображение ошибок пользователю: toast/alert рядом с формой вместо console.error.	Сейчас часть ошибок пользователь просто не увидит.
P1	Frontend: UX	Добавить loading/empty states для всех страниц и блокировку кнопок на время запроса.	Уменьшает риск двойных сохранений и делает интерфейс взрослее.
P1	Frontend: формы	Добавить клиентскую валидацию обязательных полей и подсказки формата code: REQ-001, ENT-001 и т.п.	Снижает число ошибок и помогает демо-сценарию.
P1	Frontend: навигация	Добавить route /projects/:projectId/artifacts/:artifactId для карточки артефакта.	Нужна нормальная навигация из таблицы, графа и анализа.
P1	Frontend: экспорт	Добавить явные кнопки скачать JSON/RDF с понятными именами файлов.	Сейчас ссылки могут просто открывать данные в браузере.
P1	Frontend: cleanup	Удалить стандартные Vite/React остатки: App.css с шаблонными стилями, react.svg, vite.svg, frontend/README шаблон.	Не влияет на работу, но влияет на впечатление от проекта.
P2	README	Исправить название на “Интерфейс управления жизненным циклом рабочего места программиста”.	Текущая формулировка может звучать как учет рабочих мест сотрудников, а не рабочее место программиста.
P2	README	Добавить раздел “Что реализовано” с честным списком функций и “Ограничения MVP”.	Помогает научруку/комиссии понять масштаб и не ждать IDE/Jira/Protégé.
P2	README	Добавить демонстрационный сценарий: что нажать и что показать на защите за 3-5 минут.	Это пригодится и для защиты, и для текста ВКР.
P2	Зависимости	Закрепить версии в backend/requirements.txt и пересохранить файл в UTF-8.	Упрощает запуск на другом компьютере.
P2	Зависимости	Проверить npm audit/build; зафиксировать npm ci как рекомендуемый способ установки.	Повышает воспроизводимость фронта.
P2	Тесты	Добавить тесты: create artifact, duplicate code, invalid type, create relation, invalid relation type, duplicate relation, cycle, graph, export.	Текущий один тест impact - хорошо, но для “идеально” мало.
P2	Демо-данные	Расширить demo: добавить минимум 2 требования, 2 модуля, файл, функцию, тест, документ, дефект/change request, проектное решение.	Сильнее показывает жизненный цикл, трассируемость и анализ влияния.
P2	Демо-данные	Добавить code fragment в demo-проект.	Чтобы редактор кода сразу было чем показать.
P2	Документация API	Добавить краткое описание endpoints или screenshots Swagger в приложение/README.	Удобно для ВКР и проверки.
P2	ВКР-формулировки	Не заявлять полноценный OWL reasoner, SPARQL, IDE, Git-интеграцию, многопользовательский режим.	Чтобы защита не превратилась в вопросы о нереализованных промышленных возможностях.

Минимальный порядок выполнения

№	Шаг
1	Сначала закрыть UI-сценарии: создание проекта, создание/редактирование/удаление артефакта, карточка артефакта, создание/удаление связей, редактор кода.
2	Потом закрыть целостность backend: уникальность code, проверка type_id/relation_type_id, запрет дублей, каскад/блокировка удаления.
3	Затем усилить анализ влияния: affects_impact, нормальный path, выбор типов связей, подсветка результата на графе.
4	После этого добить диагностику, поиск, фильтры графа, импорт JSON и расширенный RDF/Turtle.
5	В финале почистить документацию, README, зависимости, тесты и демо-сценарий для защиты.

Что можно честно оставить как ограничения MVP

- Не делать полноценную IDE, компиляцию, запуск и отладку кода.
- Не делать многопользовательский режим, роли и авторизацию.
- Не делать полноценный OWL reasoner, SPARQL endpoint и автоматический логический вывод.
- Не делать интеграцию с Git и автоматический парсинг исходного кода.
- Не делать промышленный workflow согласования требований.

Итоговая цель после правок

На защите должна демонстрироваться цепочка: проект -> артефакты -> связи -> граф -> анализ влияния -> кодовый фрагмент -> экспорт JSON/RDF. Если эта цепочка работает руками из интерфейса, приложение можно считать полноценным защищаемым MVP по теме диплома.